NISHANTHAN V
241801190

EXPERIMENT 6b – TIME SERIES DATA ANALYSIS AND FORECAST

Learning Objective

The objective of this experiment is to understand and implement the fundamental concepts of time series data analysis and forecasting using Python. The experiment involves analysing historical monthly airline passenger data, identifying trend and seasonal patterns, checking stationarity using the Augmented Dickey-Fuller test, transforming a non-stationary series using differencing, building an ARIMA forecasting model, predicting passenger numbers for the test period, and evaluating the forecasting performance using suitable error metrics.

Libraries Used / Purpose

- **Pandas** – Used to load, manipulate, and analyse the time series dataset.
- **NumPy** – Used for numerical operations.
- **Matplotlib** – Used to visualize the time series, differenced series, and forecast results.
- **Statsmodels** – Used for the Augmented Dickey-Fuller stationarity test and ARIMA model.
- **Scikit-learn** – Used to calculate MSE, MAE, and RMSE evaluation metrics.

Program Code

```
import pandas as pd import

numpy as np import

matplotlib.pyplot as plt



from statsmodels.tsa.stattools import adfuller from

statsmodels.tsa.arima.model import ARIMA



from sklearn.metrics import mean_squared_error, mean_absolute_error


url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
```

```
df = pd.read_csv(
    url,
    parse_dates=['Month'],
index_col='Month'
)

print("--- First 5 Records ---")
display(df.head())

print("\n--- Last 5 Records ---")
display(df.tail())

print("\n--- Dataset Shape ---")
print(df.shape)

print("\n--- Data Types ---")
print(df.dtypes)

print("\n--- Dataset Information ---")
df.info()

print("\n--- Missing Values ---")
print(df.isnull().sum()) print("\n---
Summary Statistics ---")
display(df.describe())
```

```
plt.figure(figsize=(12, 6))


plt.plot(
df.index,
df['Passengers']
)


plt.title('Monthly Airline Passenger Data')
plt.xlabel('Year') plt.ylabel('Number of
Passengers') plt.grid(True) plt.show()


def adf_test(series, name):


    result = adfuller(
series.dropna(),         autolag='AIC'
    )


    print("\n--- ADF Test:", name, "---")
print("ADF Statistic:", result[0])
print("p-value:", result[1])
print("Number of Lags:", result[2])
print("Number of Observations:",
result[3])
```

```
    print("\nCritical Values:")

    for key, value in result[4].items():
        print(f"{key}: {value}")

    if result[1] <= 0.05:
        print("\nResult: The series is stationary.")
else:
        print("\nResult: The series is non-stationary.")

    return result

adf_original = adf_test(
df['Passengers'],
    "Original Series"
)

df['Passengers_Diff'] = (
df['Passengers'].diff()
)

print("\n--- Differenced DataFrame ---")
display(df.head(10))
plt.figure(figsize=(12, 6))
```

```
plt.plot(     df.index,
df['Passengers_Diff']
)

plt.title('First Differenced Airline Passenger Series')
plt.xlabel('Year') plt.ylabel('Differenced
Passengers') plt.grid(True) plt.show()

adf_diff = adf_test(
df['Passengers_Diff'],
    "First Differenced Series"
)

d = 1

if adf_diff[1] > 0.05:

    df['Passengers_Diff2'] = (
df['Passengers_Diff'].diff()
    )

    print("\n--- Second Differenced DataFrame ---")
display(df.head(10))

    plt.figure(figsize=(12, 6))
```

```
    plt.plot(        df.index,
df['Passengers_Diff2']
   )

   plt.title('Second Differenced Airline Passenger Series')
plt.xlabel('Year')    plt.ylabel('Second Difference')
plt.grid(True)    plt.show()

   adf_diff2 = adf_test(
df['Passengers_Diff2'],
       "Second Differenced Series"
   )

   d = 2

series = df['Passengers']

train_size = int(
len(series) * 0.80
)

train = series.iloc[:train_size] test
= series.iloc[train_size:]
```

```
print("\n--- Train/Test Split ---")
print("Training samples:", len(train))
print("Testing samples:", len(test))

print("\nTraining Period:") print(train.index.min(),
"to", train.index.max())

print("\nTesting Period:") print(test.index.min(),
"to", test.index.max())

model = ARIMA(
    train,
order=(2, d, 2)
)

model_fit = model.fit()

print("\n--- ARIMA Model Summary ---")
print(model_fit.summary())

forecast = model_fit.forecast(
steps=len(test)
)
```

```
forecast = pd.Series(
forecast,     index=test.index
)

mse = mean_squared_error(
    test,
forecast
)

mae = mean_absolute_error(
    test,
forecast
)

rmse = np.sqrt(mse)

print("\n--- Forecast Evaluation Metrics ---")
print("MSE :", mse) print("MAE :", mae)
print("RMSE:", rmse)

forecast_df = pd.DataFrame({
    'Actual': test,
    'Predicted': forecast
})
```

```
print("\n--- Actual vs Predicted Passengers ---") display(forecast_df)


plt.figure(figsize=(14, 7))


plt.plot(     train.index,
train,     label='Training
Data'
)


plt.plot(     test.index,
test,     label='Actual Test
Data'
)


plt.plot(
forecast.index,
forecast,
label='ARIMA
Forecast'

)


plt.title('Airline Passenger Forecast using ARIMA')
plt.xlabel('Year') plt.ylabel('Number of
Passengers') plt.legend() plt.grid(True) plt.show()
```

Learning Outcome

After completing this experiment, students will be able to load and explore monthly time series data using Pandas, visualize historical trends and seasonal patterns, perform the Augmented Dickey-Fuller test to determine stationarity, transform a non-stationary series into a stationary series using differencing, determine the appropriate differencing order for an ARIMA model, divide the original time series into training and testing datasets, build an ARIMA forecasting model, generate predictions for the test period, evaluate forecasting performance using MSE, MAE, and RMSE, and visualize the actual and predicted passenger values together.